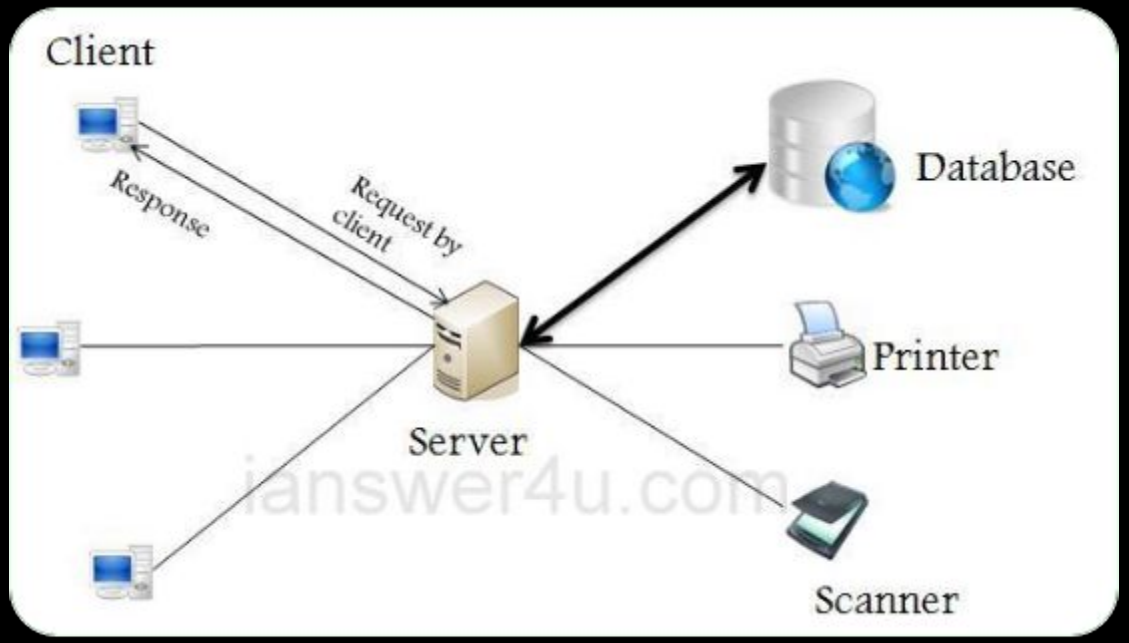
Sistema Cliente-Servidor

Redes de Computadores e Sistemas Distribuídos

Arquitetura e Protocolo

Arquitetura do Sistema

- **Modelo:** Aplicação Cliente-Servidor com conexão TCP persistente (porta 6789 padrão).
- **Servidor:** Aceita conexões, processa requisições, mantém estado compartilhado e controla concorrência/falhas.
- **Cliente:** Inicia conexão, envia comandos sequenciais e é identificado pelo soquete.
- **Persistência:** A conexão permanece ativa, permitindo múltiplas operações até o encerramento manual ou falha de rede.



Protocolo e Armazenamento

- **Comunicação:** O cliente envia uma linha terminada em \n. A primeira palavra determina a operação.
- **Respostas:** O servidor envia frases curtas sinalizando o status da execução (sucesso ou explicação de erro).
- **Estrutura de Dados:** Recursos manipulados via struct (id, value, reserved, reserved_by).
- **Identificação Única:** O descritor sockfd atua exclusivamente como o ID de quem reservou o recurso.

Formato das Requisições

Comando	Formato	Descrição
CREATE	CREATE	Cria recurso com identificador e valor
GET	GET	Consulta o valor de um recurso
SET	SET	Atualiza o valor de um recurso
RESERVE	RESERVE	Reserva um recurso livre
RELEASE	RELEASE	Libera um recurso reservado
LIST	LIST	Lista todos os recursos e seus estados
EXIT	EXIT	Encerra a conexão com o servidor

Formato das Respostas

Resposta do Servidor	Situação
resource created	Sucesso no comando CREATE
resource with id X has value Y	Sucesso no comando GET
value changed	Sucesso no comando SET
resource reserved	Sucesso no comando RESERVE
resource released	Sucesso no comando RELEASE
id: X | value: Y | reserved: Z	Retorno unitário (linha) no comando LIST
no resources	LIST com nenhum recurso previamente criado
ERROR:	Qualquer operação inválida, falha ou negação de acesso

Concorrência e Tratamento de Falhas

Gerenciamento do Servidor

Concorrência

- Implementação nativa em **Threads** utilizando a biblioteca pthread.h.
- Cada conexão detectada instância uma thread *'parser'* exclusiva para aquele soquete.
- Utilização restrita de um **Mutex Global**, assegurando que operações críticas sobre os recursos ocorram de forma atômica.

Tratamento de Falhas

- Validação de más sintaxes, valores nulos e proteção contra acessos a recursos reservados por terceiros.
- **Etapa 1:** Registro persistente e seguro através da função logMessage() no back-end.
- **Etapa 2:** Disparo imediato de pacote send() contendo diagnóstico conciso para a tela do cliente.

Experimentos e Avaliação

Bateria de Testes

Teste Simples

Avaliação unitária com 1 cliente testando todo o rol de operações. Resultado plenamente satisfatório e fluxo ideal validado.

Concorrência

Gargalo forçado com 2 clientes executando RESERVE na mesma ID. O sistema evitou colisão e disparou erro para a thread secundária.

Auto-Release / Falha

Desconexão abrupta testada via kill no terminal. O servidor identificou, realizou o auto-release do recurso e registrou em log.

Teste de Carga

Escala simultânea de 10 a 30 clientes conectados de maneira assíncrona por FIFO. Ausência completa de deadlocks detectados.

Limitações e Restrições

- **Funcionalidades:** Inexistência do comando DELETE (persistência ininterrupta) e ausência de logging no lado do cliente para poupar overhead.
- **Protocolo e Redes:** A segurança da autenticação é frágil em produção por basear-se no sockfd. O framing isolado por \n sob o protocolo TCP pode sofrer fragmentação em rede instável.
- **Limites de Estrutura:** A resposta do comando LIST é truncada silenciosamente em 1000 bytes. O teto máximo na memória fica cravado no MAXRESOURCES=100 em etapa de compilação.
- **Falsos Positivos de Latência:** A bateria de testes ocorreu em modelo de loopback na mesma interface (i5-13420H), mascarando atrasos orgânicos de propagação, perda de pacotes e gargalo do processador sob carga contenciosa real.